Generador de números aleatorios cuántico

Genesis Silesky

20 de octubre de 2026

1. Generador de números aleatorios cuántico (QRNG)

1.1. Objetivo

Generar bits aleatorios midiendo qubits en superposición y usarlos para elegir la jugada de un bot de Tic-Tac-Toe, señalando las limitaciones de la simulación.

1.2. Conceptos previos

Concepto previo: aleatoriedad, uniformidad y módulo

- **Aleatoriedad pseudoaleatoria vs. física:** un PRNG es un algoritmo determinista a partir de una semilla; una fuente física (p. ej. la medición de un qubit real) no lo es.
- **Uniformidad:** todos los resultados posibles tienen igual probabilidad.
- Medición y regla de Born (capítulo 04).

1.3. Fundamento teórico

$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: medir da 0 o 1 con probabilidad $\frac{1}{2}$. Con n qubits en paralelo se obtienen n bits independientes $\rightarrow$ un entero uniforme en $\{0, \ldots, 2^n - 1\}$.

Sesgo por módulo. Para elegir entre m opciones con x mód m la distribución solo es uniforme si m divide a 2^n. En el código se eligen 4 bits (16 valores) para $m = 5$ casillas; $16 = 3 \cdot 5 + 1$, así que la casilla 0 sale con probabilidad $4/16$ y las demás con $3/16$. Una solución es el *muestreo por rechazo*: descartar los valores ≥ 15 y repetir.

La advertencia importante. `AerSimulator` calcula las probabilidades y luego toma muestras con un generador pseudoaleatorio. Por lo tanto, **en simulación los bits no son aleatoriedad física**. Solo ejecutando en hardware real la fuente es física (aun así, el ruido del equipo puede introducir sesgo y se recomienda post-procesamiento/extractores).

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 09 - Aleatoriedad cuantica (QRNG), conecta con el Tic-Tac-Toe

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

```

```python
# %% Celda 2: generador de bits
def bits_cuanticos(n_bits: int) -> str:
    """Devuelve una cadena de n_bits bits generada midiendo n_bits qubits en
    superposicion. Con H, cada qubit da 0 o 1 con probabilidad 1/2."""
    qc = QuantumCircuit(n_bits, n_bits)
    qc.h(range(n_bits))                        # cada qubit en (|0> + |1>)/sqrt(2)
    qc.measure(range(n_bits), range(n_bits))
    # memory=True guarda el resultado de CADA shot, no solo el conteo.
    # Con shots=1 obtenemos una unica cadena, p.ej. '11011110'.
    return AerSimulator().run(qc, shots=1, memory=True).result().get_memory()[0]

# %% Celda 3: usar los bits
bits = bits_cuanticos(8)
print("Bits:", bits, "-> entero:", int(bits, 2))  # int(..., 2) lee base 2

# Elegir una casilla libre del tablero 3x3 (indices 0-8).
# El modulo (%) mapea el entero a un indice valido de la lista.
# Nota: 16 valores % 5 casillas no es perfectamente uniforme (leve sesgo);
# para un uso serio se descartan los valores sobrantes y se repite.
libres = [0, 2, 3, 5, 8]
print("Jugada del bot:", libres[int(bits_cuanticos(4), 2) % len(libres)])

# IMPORTANTE: AerSimulator usa un generador pseudoaleatorio. Solo en
# hardware real la aleatoriedad es de origen fisico.
```

1.5. Explicación celda por celda

Celda 2 La función arma un circuito con n qubits, aplica H a todos y mide. `shots=1, memory=True` devuelve la cadena de esa única ejecución.

Celda 3 Convierte los bits en un entero (`int(bits, 2)`) y elige una casilla libre del tablero con el operador módulo.

1.6. Salida obtenida

```
Bits: 10100010 -> entero: 162
Jugada del bot: 3
```

Distinta en cada ejecución (es el punto del generador).

1.7. Interpretación

Se ilustra el vínculo entre el concepto (medición) y una aplicación concreta (un bot que elige jugadas). El ejemplo también expone una buena práctica de ingeniería: validar que el mapeo de bits a opciones no introduzca sesgo.

1.8. Ejercicios

1. Implemente el muestreo por rechazo y verifique con 10 000 jugadas que las 5 casillas tienen frecuencia $\approx 20\,\%$.
2. Mida con una prueba de χ^2 la uniformidad de 10 000 bits.
3. Cambie el simulador por un backend real (sin ejecutar en la charla) y compare.